

Django Paginator -- Complete Notes

1. Django Paginator ?

Django- `Paginator` built-in class, `**QuerySet list- page- **`

, e-commerce website- 100 product , :

```
```text
```

Page 1 10 products

Page 2 10 products

Page 3 10 products

...

```
```
```

Import:

```
```python
```

```
from django.core.paginator import Paginator
```

```
```
```

```
---
```

2. Basic Workflow

Paginator workflow:

```
```text
```

QuerySet / List

Paginator()

page\_number

get\_page()

page\_obj

Template

...

---

# 3. Paginator

50 product :

```
```python
```

```
products = Product.objects.all()
```

```
paginator = Paginator(products, 10)
```

```
...
```

```
:
```

```
```python
```

```
Paginator(products, 10)
```

```
...
```

```
:
```

> page- 10 product

50 product :

```
```text
```

Page 1 10

Page 2 10

Page 3 10

Page 4 10

Page 5 10

...

5 page

4. Paginator- Basic Syntax

```
```python
```

```
Paginator(object_list, per_page)
```

```
```
```

```
#### `object_list`
```

QuerySet list paginate

Example:

```
```python
```

```
Product.objects.all()
```

```
```
```

```
#### `per_page`
```

page- item

Example:

```
```python
```

```
10
```

```
```
```

5. `page` Parameter

URL page number :

```
```python
page_number = request.GET.get("page")
```
```

URL:

```
```text
/products/?page=2
```
```

:

```
```python
page_number
```
```

value :

```
```text
"2"
```
```

6. `get\_page()`

Page number :

```
```python
page_obj = paginator.get_page(page_number)
```
```

`page\_obj` page- object

Example:

```
```python
paginator = Paginator(products, 10)

page_number = request.GET.get("page")

page_obj = paginator.get_page(page_number)
```
```

:

```
```text
```

?page=2

```
```
```

, `page\_obj` page 2- products

```
---
```

7. `get\_page()` Useful?

`get\_page()` invalid missing page number- convenient behavior

:

```
```text
```

?page=abc

...

:

```text

?page=9999

...

input valid page return

strict validation , `page()` method `PageNotAnInteger` `EmptyPage` handle

8. `Paginator` - Important Parameters

Basic:

```python

Paginator(object\_list, per\_page)

...

parameter:

```python

Paginator(

object_list,

per_page,

orphans=0

)

...

Django- API- `allow\_empty\_first\_page` option

9. `orphans` ?

`orphans` page- item

Example:

```
```python
```

```
products = Product.objects.all()
```

```
paginator = Paginator(
```

```
products,
```

```
10,
```

```
orphans=2
```

```
)
```

```
```
```

```
:
```

```
```text
```

```
per_page = 10
```

```
orphans = 2
```

```
```
```

```
:
```

```
> page- 2    item    page- merge
```

```
### Example
```

```
:
```

```
```text
```

Total products = 22

per\_page = 10

orphans = 2

...

Normally:

```text

Page 1 10

Page 2 10

Page 3 2

...

`orphans=2` :

```text

Page 1 10

Page 2 12

...

2-item- page

---

# 10. Important Correction about `orphans`

`orphans` :

> " page- minimum item"

accurate

:



```
> `orphans` , page- item previous page- merge
```

---

## # 11. Important `page\_obj` Attributes

`page\_obj`- useful attribute

| Attribute                                      |                                         |
|------------------------------------------------|-----------------------------------------|
| <code>`page_obj.number`</code>                 | Current page number                     |
| <code>`page_obj.paginator`</code>              | Parent Paginator                        |
| <code>`page_obj.paginator.num_pages`</code>    | Total number of pages                   |
| <code>`page_obj.has_next`</code>               | Next page                               |
| <code>`page_obj.has_previous`</code>           | Previous page                           |
| <code>`page_obj.next_page_number()`</code>     | Next page number                        |
| <code>`page_obj.previous_page_number()`</code> | Previous page number                    |
| <code>`page_obj.start_index()`</code>          | Current page- first item- overall index |
| <code>`page_obj.end_index()`</code>            | Current page- last item- overall index  |

---

## # 12. FBV Pagination

```
views.py
```

```
```python
```

```
from django.shortcuts import render
```

```
from django.core.paginator import Paginator
```

```
from .models import Product
```

```

def product_list(request):

    products = Product.objects.all()

    paginator = Paginator(
        products,
        10
    )

    page_number = request.GET.get("page")

    page_obj = paginator.get_page(
        page_number
    )

    return render(
        request,
        "products.html",
        {
            "page_obj": page_obj
        }
    )

```

```

'''

```

```

'''

```

13. URL

```

```python

```

```

from django.urls import path

```

```

from . import views

```

```
urlpatterns = [
 path(
 "products/",
 views.product_list,
 name="product_list"
),
]
```

```

```

# 14. Template

```
products.html
```

```
``html
```

Products

```
{% for product in page_obj %}
```

```
{{ product.name }}
```

```
-
```

```
${{ product.price }}
```

```
{% empty %}
```

No products found.

```
{% endfor %}
```

```
{% if page_obj.has_previous %}
```

Previous

```
{% endif %}
```

Page {{ page\_obj.number }}

of

{{ page\_obj.paginator.num\_pages }}

```
{% if page_obj.has_next %}
```

Next

```
{% endif %}
```

...

---

# 15. URL Example

```
```text
```

/products/

...

Page 1

```
```text
```

/products/?page=2

...

Page 2

```text

/products/?page=3

...

Page 3

16. How Normal Pagination Works

```text

Browser

/products/?page=2

Django View

Product.objects.all()

Paginator(products, 10)

get\_page(2)

page\_obj

Template

Browser

...

---

## # 17. CBV + Manual Paginator

CBV    `ProductListView` class file-

original notes- :

```
```python
```

```
class ProductListView(ListView):
```

```
...
```

```
```
```

```
:
```

```
```python
```

```
class ProductListView(View):
```

```
...
```

```
```
```

class class- overwrite

```

```

## # 18. CBV + Generic `ListView` Pagination

Django- pagination-    CBV implementation    `ListView`

```
```python
```

```
from django.views.generic import ListView
```

```
from .models import Product
```

```
class ProductListView(ListView):
```

```
model = Product
```

```
template_name = "products.html"
```

```
context_object_name = "products"
```

```
paginate_by = 10
```

```
...
```

Django internally pagination handle

manually:

```
``python
```

```
Paginator(...)
```

```
...
```

```
---
```

```
# 19. `paginate_by`
```

```
``python
```

```
paginate_by = 10
```

```
...
```

```
:
```

```
> page- 10 object
```

```
`ListView`- pagination configuration
```

```
---
```

```
# 20. ListView Pagination Context
```

```
`ListView` pagination    template- :
```

```
``text
```

```
page_obj
```

```
paginator
```

```
is_paginated
```

```
object_list
```

```
``
```

```
`context_object_name` :
```

```
``python
```

```
context_object_name = "products"
```

```
``
```

```
:
```

```
``text
```

```
products
```

```
``
```

```
-
```

```
---
```

```
# 21. CBV Template
```

```
``html
```

```
Products
```

```
{% for product in products %}
```


{{ product.name }}

-

\${{ product.price }}

{% empty %}

No products found.

{% endfor %}

{% if is_paginated %}

{% if page_obj.has_previous %}

Previous

{% endif %}

Page {{ page_obj.number }}

of

{{ paginator.num_pages }}

{% if page_obj.has_next %}

Next

{% endif %}

```
{% endif %}
```

```
...
```

```
---
```

22. CBV Pagination Flow

```
```text
```

ListView

model = Product

paginate\_by = 10

Django Pagination

page\_obj

Template

```
...
```

manual `Paginator` implementation- cleaner

```

```

# 23. Manual CBV Pagination

```
`View` class pagination :
```

```
```python
```

```
from django.views import View
```

```
from django.shortcuts import render
```

```
from django.core.paginator import Paginator
```

```
from .models import Product

class ProductListView(View):

    def get(self, request):

        products = Product.objects.all()

        paginator = Paginator(
            products,
            10
        )

        page_number = request.GET.get("page")

        page_obj = paginator.get_page(
            page_number
        )

        context = {
            "page_obj": page_obj
        }

        return render(
            request,
            "products.html",
            context
        )
...

valid
```

simple list pagination- `ListView` + `paginate\_by` code

24. Pagination with Search / Filter

important

URL:

```
```text
```

```
/products/?search=phone&page=2
```

```
```
```

`page` link :

```
```html
```

```
```
```

click `search=phone`

existing query parameters

25. Query Parameters Preserve

Template-:

```
```html
```

Next

```
```
```

multiple GET parameters manually URL cumbersome

robust approach query string- existing values preserve

26. Search + Pagination Example

views.py

```
```python
```

```
from django.views.generic import ListView
```

```
from .models import Product
```

```
class ProductListView(ListView):
```

```
 model = Product
```

```
 template_name = "products.html"
```

```
 context_object_name = "products"
```

```
 paginate_by = 10
```

```
 def get_queryset(self):
```

```
 queryset = super().get_queryset()
```

```
 search = self.request.GET.get("search")
```

```
 if search:
```

```
 queryset = queryset.filter(
```

```
 name__icontains=search
```

```
)
```

return queryset

...

URL:

```text

/products/?search=phone&page=2

...

pagination- search filter apply

27. AJAX Pagination

Normal pagination-:

```text

Click Next

Browser request

Entire page reload

...

AJAX pagination-:

```text

Click Next

AJAX request

Server returns partial HTML

Only product container update

No full page reload

```

---

# 28. AJAX Request Detect

Django-:

```python

request.is_ajax()

```

method

Modern Django- :

```python

request.headers.get("x-requested-with") == "XMLHttpRequest"

```

---

# 29. AJAX + Manual Paginator

## views.py

```python

from django.views import View

from django.shortcuts import render

from django.core.paginator import Paginator

from .models import Product

```
class ProductListView(View):

    def get(self, request):

        products = Product.objects.all()

        paginator = Paginator(
            products,
            10
        )

        page_number = request.GET.get("page")

        page_obj = paginator.get_page(
            page_number
        )

        context = {
            "page_obj": page_obj
        }

        if request.headers.get(
            "x-requested-with"
        ) == "XMLHttpRequest":

            return render(
                request,
                "partials/product_list.html",
                context
            )
```



```
return render(  
    request,  
    "products.html",  
    context  
)  
---
```

30. Main Template

```
## products.html
```

```
```html
```

Products

```
{% include "partials/product_list.html" %}
```

```

```

```
:
```

```
```html
```

```
---
```

content AJAX replace

```
---
```

31. Partial Template

File:

```
``text

templates/
products.html

partials/
product_list.html
...

## partials/product_list.html

``html

{% for product in page_obj %}

{{ product.name }}

-

${{ product.price }}

{% empty %}

No products found.

{% endfor %}

{% if page_obj.has_previous %}

button

class="page-btn"

data-page="{{ page_obj.previous_page_number }}"

>
```

Previous

```
{% endif %}
```

```
Page {{ page_obj.number }}
```

```
of
```

```
{{ page_obj.paginator.num_pages }}
```

```
{% if page_obj.has_next %}
```

```
button
```

```
class="page-btn"
```

```
data-page="{{ page_obj.next_page_number }}"
```

```
>
```

Next

```
{% endif %}
```

```
...
```

```
---
```

32. AJAX Script

Modern JavaScript jQuery AJAX pagination

```
```html
```

```
document.addEventListener("click", function(event) {
```

```
if (!event.target.classList.contains("page-btn")) {
```

```

return;
}

const page = event.target.dataset.page;

fetch(
 `${% url 'product_list' %}?page=${page}`,
 {
 headers: {
 "X-Requested-With": "XMLHttpRequest"
 }
 }
)
.then(response => response.text())
.then(data => {

document
.getElementById("product-container")
.innerHTML = data;

});

});

...

:

```text
X-Requested-With: XMLHttpRequest
...

```

header , Django AJAX request

33. AJAX Pagination Flow

``text

Initial Request

products.html

product-container

Partial included

Products displayed

...

:

``text

User clicks Next

JavaScript

fetch()

/products/?page=2

AJAX header

Django View

Paginator

partials/product_list.html

HTML response

#product-container

innerHTML update

...

page reload

34. Partial Template ?

Main page:

```text

products.html

...

page structure

Partial:

```text

partials/product_list.html

...

dynamic product list + pagination

AJAX request- HTML page

:

```text

product list

+

pagination controls

...

response implementation

---

## # 35. AJAX Pagination-

AJAX pagination- `Previous` `Next` button AJAX response-

:

```
```javascript
```

```
document.addEventListener("click", ...)
```

```
```
```

event delegation

AJAX- `.page-btn` elements event handler

---

## # 36. jQuery AJAX Pagination

jQuery :

```
```html
```

```
$(document).on("click", ".page-btn", function() {
```

```
const page = $(this).data("page");
```

```
$.ajax({

url: "{% url 'product_list' %}",

data: {

page: page

},

headers: {

"X-Requested-With": "XMLHttpRequest"

},

success: function(data) {

$("#product-container").html(data);

}

});

});

...

valid

---
```

37. AJAX Normal Pagination

	Normal	AJAX	
	-----	-----	
	Page reload		
	Implementation		complex

| JavaScript | | |

| UX | Standard | More dynamic |

| Server response | Full page | Partial HTML |

| URL page number | address bar- | manage |

38. `Paginator` vs `ListView`

Manual Paginator

```
```python
```

```
paginator = Paginator(
```

```
products,
```

```
10
```

```
)
```

```
page_number = request.GET.get("page")
```

```
page_obj = paginator.get_page(
```

```
page_number
```

```
)
```

```
```
```

ListView

```
```python
```

```
class ProductListView(ListView):
```

```
model = Product
```

```
paginate_by = 10
```

```
```
```

?

``text

Simple list pagination

ListView + paginate_by

Custom pagination logic

Paginator manually

39. Pagination + `get\_queryset()`

CBV- filter pagination

``python

```
class ProductListView(ListView):
```

```
    model = Product
```

```
    template_name = "products.html"
```

```
    context_object_name = "products"
```

```
    paginate_by = 10
```

```
    def get_queryset(self):
```

```
        queryset = super().get_queryset()
```

```
        category = self.request.GET.get(
```

```
            "category"
```

```

)

if category:
    queryset = queryset.filter(
        category=category
    )

return queryset
'''

:

```text
get_queryset()

Filter

paginate_by

Pagination
'''

pagination filtered queryset-

40. Pagination + `select_related()`

ForeignKey:

```python
class ProductListView(ListView):

    model = Product

```

```
paginate_by = 10
```

```
def get_queryset(self):
```

```
    return (
```

```
        super()
```

```
        .get_queryset()
```

```
        .select_related("category")
```

```
    )
```

```
    ...
```

template-:

```
``django
```

```
{{ product.category.name }}
```

```
...`
```

unnecessary database queries

```
---
```

41. Common Mistakes

Mistake 1 -- class

```
``python
```

```
class ProductListView(ListView):
```

```
    ...
```

```
class ProductListView(View):
```

```
    ...
```

```
...`
```

approach choose

Mistake 2 -- `request.is\_ajax()`

approach:

```
```python
if request.is_ajax():
 ...
```

Modern Django- :

```
```python
if request.headers.get(
    "x-requested-with"
) == "XMLHttpRequest":
    ...
```

Mistake 3 -- AJAX request- full page return

AJAX pagination- :

```
```python
return render(
 request,
 "products.html",
 context
)
```

```
'''
```

page HTML

partial:

```
'''python
```

```
return render(
```

```
request,
```

```
"partials/product_list.html",
```

```
context
```

```
)
```

```
'''
```

```
return
```

```

```

```
Mistake 4 -- `page_obj`
```

Template-:

```
'''django
```

```
{% for product in page_obj %}
```

```
'''
```

view :

```
'''python
```

```
{
```

```
"page_obj": page_obj
```

```
}
```

```
'''
```

---

## # 42. `Paginator` Important Methods

`Paginator` object:

```
```python
paginator = Paginator(
products,
10
)
```
```

useful attributes/methods:

```
```text
paginator.count
paginator.num_pages
paginator.page_range
paginator.get_page()
paginator.page()
```
```

### `count`

object

```
```python
paginator.count
```
```

```
`num_pages`
```

page

```
``python
```

```
paginator.num_pages
```

```
...
```

```
`page_range`
```

Page number- range

```
``python
```

```
paginator.page_range
```

```
...
```

:

```
``text
```

```
1, 2, 3, 4, 5
```

```
...
```

```

```

```
43. `Page` Object Important Methods
```

```
``python
```

```
page_obj.has_next()
```

```
page_obj.has_previous()
```

```
page_obj.next_page_number()
```

```
page_obj.previous_page_number()
```

```
...
```



Template- Django automatically callable attribute handling :

```
``django
{% if page_obj.has_next %}
...

:

``django
{{ page_obj.next_page_number }}
...

```

# 44. Complete Mental Model

```
``text
Product.objects.all()

Paginator

per_page = 10

request.GET["page"]

paginator.get_page()

page_obj
```

Product List Pagination

Template

...

---

# 45. Final Summary

#### Manual Pagination

```python

from django.core.paginator import Paginator

paginator = Paginator(

products,

10

)

page_number = request.GET.get("page")

page_obj = paginator.get_page(

page_number

)

...

CBV Pagination

```python

class ProductListView(ListView):

model = Product

paginate\_by = 10

...

### Main Page

```
``django
```

```
{% include "partials/product_list.html" %}
```

```
...
```

### AJAX Detection

```
``python
```

```
request.headers.get(
```

```
"x-requested-with"
```

```
) == "XMLHttpRequest"
```

```
...
```

### Pagination Attributes

```
``text
```

```
page_obj.number
```

```
page_obj.has_next
```

```
page_obj.has_previous
```

```
page_obj.next_page_number
```

```
page_obj.previous_page_number
```

```
page_obj.paginator.num_pages
```

```
...
```

### `orphans`

```
``python
```

```
Paginator(
```

```
products,
```

10,

orphans=2

)

...

page- 2    item   previous page-   merge

---

# 46. Quick Memory Trick

```text

Paginator

object_list

per_page

page_number

get_page()

page_obj

Template

...

CBV- :

```text

ListView

paginate\_by = 10

Django handles pagination

page\_obj

Template

...

AJAX- :

```text

Click

AJAX / fetch()

?page=2

Django

Partial HTML

container.innerHTML

No full page reload

...

**** : **** Django project- pagination `ListView + paginate\_by` approach pagination- custom filtering, search, AJAX dynamic behavior , `get\_queryset()` manual `Paginator`/partial template